

STORE PROCEDURES EN SQL SERVER

Procedimientos almacenados para crear, reutilizar y proteger la lógica del negocio en la base de datos.



1. ¿QUÉ ES UN STORE PROCEDURE?

Un Store Procedure (procedimiento almacenado) es un conjunto de instrucciones SQL compiladas que se almacenan en la base de datos. Se ejecutan bajo un nombre y pueden recibir parámetros, devolver resultados y controlar la lógica del negocio.



Idea clave: Permiten encapsular la lógica, mejorar el rendimiento, la seguridad y facilitar el mantenimiento del código SQL.

2. ¿PARA QUÉ SIRVE?

- ✓ Reutilizar código SQL.
- ✓ Mejorar el rendimiento (plan de ejecución almacenado).
- ✓ Reducir el tráfico de red.
- ✓ Aumentar la seguridad (permisos controlados).
- ✓ Facilitar el mantenimiento y la escalabilidad.
- ✓ Encapsular reglas de negocio complejas.
- ✓ Manejar transacciones y errores de forma controlada.



3. ¿CUÁNDO UTILIZARLOS?



Cuando la lógica se reutiliza frecuentemente.



Cuando se requiere mejor rendimiento.



Cuando necesitamos controlar el acceso a los datos.



Cuando la lógica es compleja y debe centralizarse.



Cuando varios sistemas/aplicaciones acceden a la misma lógica.

4. SINTAXIS BÁSICA

```
CREATE PROCEDURE NombreProcedimiento
    @Parametro1 TipoDato,
    @Parametro2 TipoDato = ValorPorDefecto
AS
BEGIN
    -- Instrucciones SQL
END;
```



Ejemplo simple (sin parámetros)

```
CREATE PROCEDURE sp_ObtenerEmpleados
AS
BEGIN
    SELECT IdEmpleado, Nombre, Cargo, Salario
    FROM Empleados;
END;
```



5. PARÁMETROS

TIPOS DE PARÁMETROS

IN	Recibe un valor de entrada.
OUT	Devuelve un valor de salida.
INOUT	Recibe un valor y devuelve otro.

Ejemplo con parámetros

```
CREATE PROCEDURE sp_BuscarEmpleado
    @IdEmpleado INT,
    @Nombre NVARCHAR(100) OUTPUT
AS
BEGIN
    SELECT @Nombre = Nombre
    FROM Empleados
    WHERE IdEmpleado = @IdEmpleado;
END;
```



6. EJEMPLOS PRÁCTICOS

Ejemplo 1: Procedimiento sin parámetros

```
CREATE PROCEDURE sp_ListarClientes
AS
BEGIN
    SELECT IdCliente, Nombre, Ciudad
    FROM Clientes;
END;
```



Ejemplo 2: Con parámetro de entrada (IN)

```
CREATE PROCEDURE sp_ClientesPorCiudad
    @Ciudad NVARCHAR(100)
AS
BEGIN
    SELECT * FROM Clientes
    WHERE Ciudad = @Ciudad;
END;
```



Ejemplo 3: Con parámetro de salida (OUT)

```
CREATE PROCEDURE sp_ContarEmpleados
    @Total INT OUTPUT
AS
BEGIN
    SELECT @Total = COUNT(*) FROM Empleados;
END;
```



7. EJECUTAR STORE PROCEDURE

Sin parámetros

```
EXEC sp_ObtenerEmpleados;
```



Con parámetro IN

```
EXEC sp_ClientesPorCiudad @Ciudad = 'Lima';
```



Con parámetro OUT

```
DECLARE @Total INT;
EXEC sp_ContarEmpleados @Total OUTPUT;
SELECT @Total AS TotalEmpleados;
```



8. MODIFICAR Y ELIMINAR

Modificar un Store Procedure

```
ALTER PROCEDURE sp_ObtenerEmpleados
AS
BEGIN
    SELECT IdEmpleado, Nombre, Cargo, Salario, FechaIngreso
    FROM Empleados;
END;
```



Eliminar un Store Procedure

```
DROP PROCEDURE sp_ObtenerEmpleados;
```



9. PROCEDIMIENTOS DEL SISTEMA

SQL Server incluye procedimientos almacenados del sistema que ayudan a obtener información.

- sp_help → Información de objetos
- sp_who → Usuarios conectados
- sp_helpdb → Información de bases de datos
- sp_lock → Bloqueos activos
- sp_spaceused → Espacio utilizado



10. MANEJO DE TRANSACCIONES

```
CREATE PROCEDURE sp_InsertarVenta
    @IdCliente INT,
    @Total DECIMAL(10,2)
AS
BEGIN
    BEGIN TRAN;
    INSERT INTO Ventas(IdCliente, Total, Fecha)
    VALUES(@IdCliente, @Total, GETDATE());
    COMMIT TRAN;
END;
```



11. MANEJO DE ERRORES

```
CREATE PROCEDURE sp_Division
    @Numerador INT,
    @Denominador INT
AS
BEGIN
    BEGIN TRY
        SELECT @Numerador / @Denominador AS Resultado;
    END TRY
    BEGIN CATCH
        SELECT ERROR_NUMBER() AS NumeroError,
        ERROR_MESSAGE() AS MensajeError;
    END CATCH
END;
```



12. SEGURIDAD

- Los permisos se otorgan sobre el procedimiento, no sobre las tablas.
- Permite ocultar la estructura de las tablas.
- Controla qué usuarios pueden ejecutar la lógica.



Ejemplo de permiso

```
GRANT EXECUTE ON sp_ObtenerEmpleados
TO UsuarioApp;
```

13. EJEMPLO REAL

Sistema de Ventas

Tablas involucradas:

- Clientes
- Productos
- Ventas



Procedimiento: RegistrarVenta

```
CREATE PROCEDURE sp_RegistrarVenta
    @IdCliente INT,
    @IdProducto INT,
    @Cantidad INT
AS
BEGIN
    INSERT INTO Ventas(IdCliente, IdProducto, Cantidad, Fecha)
    VALUES(@IdCliente, @IdProducto, @Cantidad, GETDATE());
END;
```

Ejecución:

```
EXEC sp_RegistrarVenta
    @IdCliente = 1,
    @IdProducto = 10,
    @Cantidad = 5;
```



14. BUENAS PRÁCTICAS

- ✓ Usar nombres descriptivos y consistentes.
- ✓ Documentar el propósito y los parámetros.
- ✓ Validar los parámetros de entrada.
- ✓ Manejar transacciones y errores correctamente.
- ✓ Evitar lógica innecesariamente compleja.
- ✓ Revisar y optimizar periódicamente.

15. RESUMEN



Store Procedure = Código SQL reutilizable, seguro y más rápido.

- ✓ Reutiliza lógica
- ✓ Mejora rendimiento
- ✓ Aumenta seguridad
- ✓ Facilita mantenimiento



USA STORE PROCEDURES PARA HACER TU BASE DE DATOS MÁS EFICIENTE, SEGURA Y ESCALABLE.

